

Modelos de Aprendizaje Supervisado

Clasificación · Regresión · QSAR

$$X \rightarrow f(X) \rightarrow y$$

X = descriptores / fingerprints

y = activo/inactivo (clasificación) o pActividad (regresión)

f = el modelo que aprende la relación

Clasificación

$y \in \{0, 1\}$ — clases discretas

¿Es activo o inactivo?

Salida: probabilidad $P(\text{activo})$

Métricas: AUC-ROC, F1, MCC

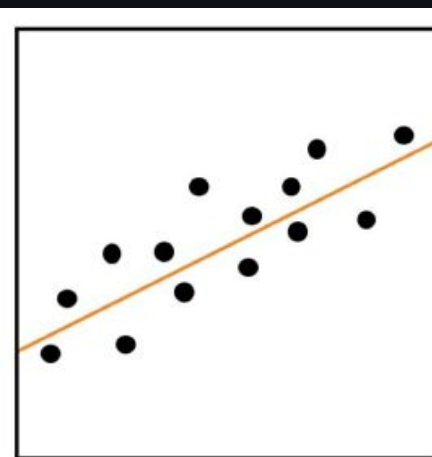
Regresión

$y \in \mathbb{R}$ — valor continuo

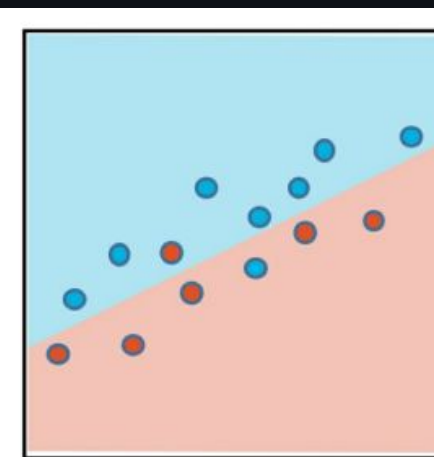
¿Cuál es la pIC50 predicha?

Salida: número real

Métricas: R^2 , RMSE, MAE



a) Regression



b) Classification

En el curso usamos clasificación como objetivo principal y regresión como objetivo secundario.

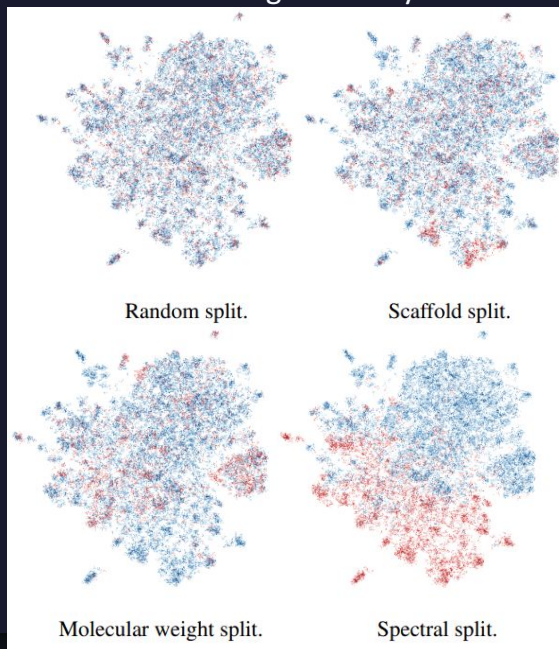
Train / Test · Muestreo y Validación Cruzada

Nunca evalúes el modelo con los mismos datos que usaste para entrenarlo — obtendrías un resultado optimista que no refleja el rendimiento real en moléculas nuevas.

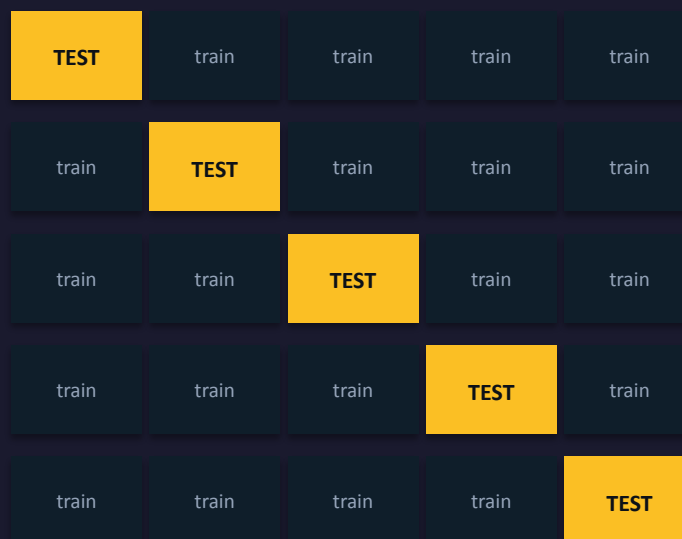
Train / Test Split

División clásica: 80% train / 20% test

- **Estratificado:** Mantiene la misma proporción de activos/inactivos en train y test. Crítico con clases desbalanceadas.
- **Scaffold split:** El test set contiene scaffolds que no aparecen en train. Evalúa si el modelo generaliza a nuevas familias químicas el escenario real en drug discovery.



Validación Cruzada k-fold



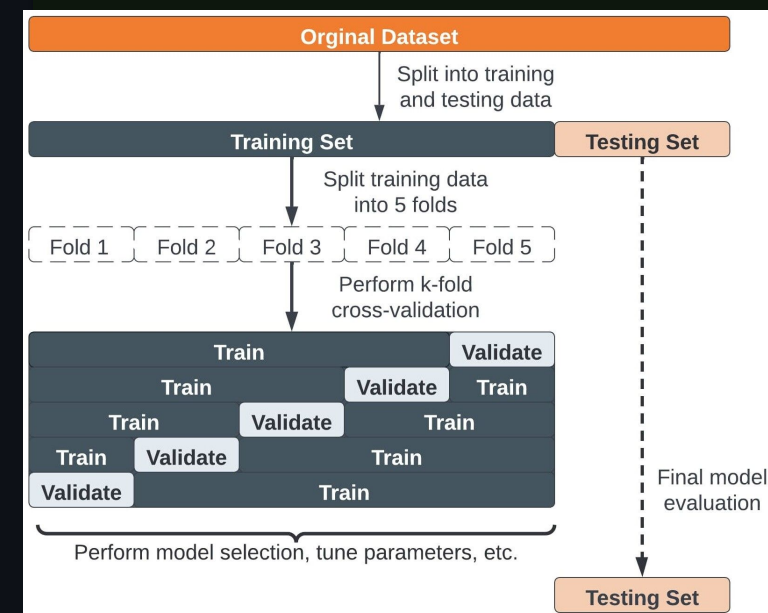
Cada fold sirve de test una vez.

Métrica final = media de 5 evaluaciones.

Ventajas: Usa todos los datos para evaluar. Estimación más robusta que un solo split. Detecta varianza alta (sobreajuste)

Stratified k-fold: Mantiene proporción de clases en cada fold “**muy recomendado con datasets desbalanceados.**”

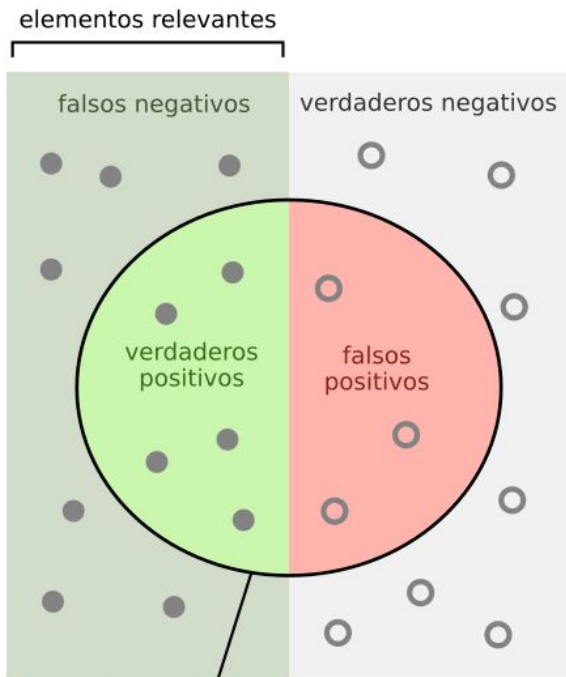
Validaciones



Ventajas:

- **Estabilidad Métrica:** Muestra con números cómo se reduce la desviación típica (el error del error).
- **Eficiencia:** Visualiza cómo pasas de evaluar solo una parte de tus datos a evaluarlos todos (100%).

Matriz de Confusión · Terminología completa



elementos seleccionados

¿Cuántos objetos relevantes se seleccionaron?
i.e. Cuántas personas enfermas son identificadas como tales.

Sensibilidad =



¿Cuántos elementos negativos se identifican como negativos?
i.e. Cuántas personas sanas son identificadas como no enfermas.

Especificidad =



VP — Verdaderos Positivos

También: éxitos

El modelo predice POSITIVO y la realidad es POSITIVO.
Activo predicho como activo.

VN — Verdaderos Negativos

También: rechazos correctos

El modelo predice NEGATIVO y la realidad es NEGATIVO.
Inactivo predicho como inactivo.

FP — Falsos Positivos

También: falsas alarmas · Error tipo I

El modelo predice POSITIVO pero la realidad es NEGATIVO.
Inactivo predicho como activo. Se sintetiza un compuesto inútil.

FN — Falsos Negativos

También: Error tipo II

El modelo predice NEGATIVO pero la realidad es POSITIVO.
Activo descartado. El coste más alto en drug discovery.

VPR — Verdaderos Positivos

Razón de Verdaderos Positivos · razón de éxitos

$$VPR = VP / (VP + FN)$$

¿De todos los activos reales, cuántos detectó el modelo?

FPR — Razón de Falsos Positivos

Falsas alarmas · fall-out

$$FPR = FP / (FP + VN)$$

¿De todos los inactivos reales, cuántos marcó el modelo como activos?

Usa en curva ROC: eje X = FPR · eje Y = Sensibilidad

Desbalance de Clases · El problema silencioso del QSAR

En ChEMBL es habitual tener 3–5 inactivos por cada activo. Si no se corrige, el modelo aprende a decir 'inactivo' siempre — y tiene accuracy alta pero es inútil.

! El modelo ingenuo

Dataset: 800 inactivos + 200 activos

Modelo que siempre dice 'inactivo':

Accuracy = $800/1000 = 80\%$ ✓

Recall activos = $0/200 = 0\%$ ✗

AUC-ROC = 0.50 ✗

F1 activos = 0.0 ✗

→ Accuracy alta, modelo inútil.

¿Qué métrica usar entonces?

AUC-ROC

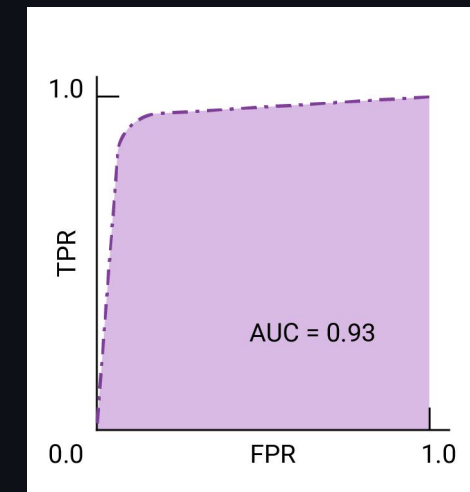
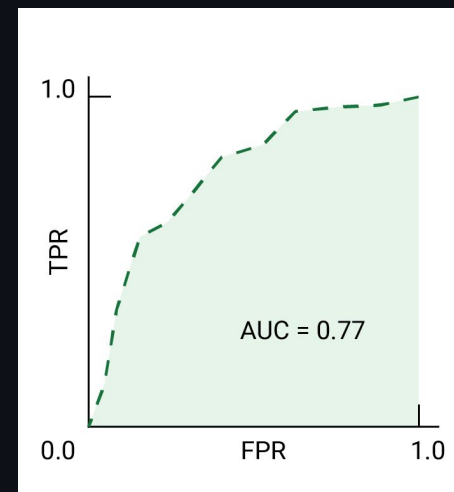
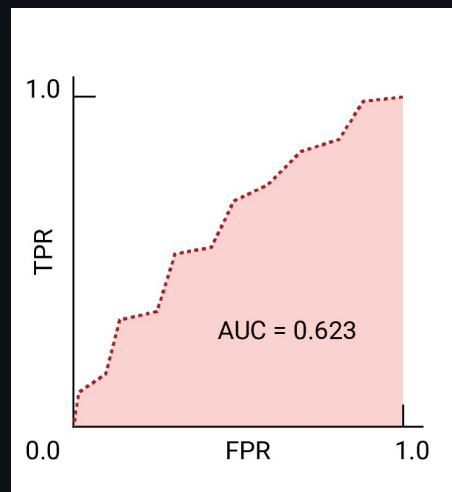
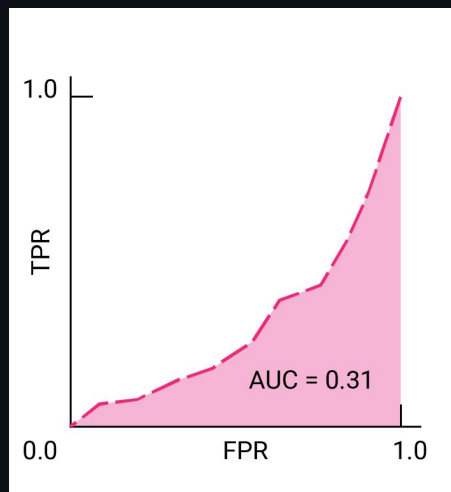
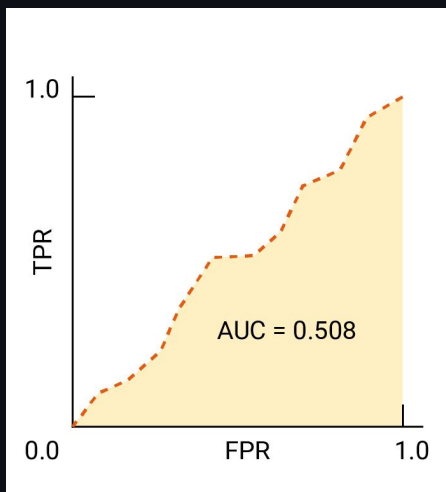
Área bajo la curva ROC.
Independiente del umbral.
Referencia: 0.5=azar,
1.0=perfecto.

F1 Score

Media armónica de precisión
y recall.
Sensible al desbalance.
Mejor que accuracy para
clases raras.

MCC

Matthews Correlation
Coefficient.
El más robusto al desbalance.
Rango $[-1, 1]$ · 0 = azar.



Métricas de Evaluación · AUC-ROC · F1 · MCC

Cada métrica responde una pregunta distinta. En drug discovery el error más costoso es descartar un activo real — eso define cuál métrica priorizar.

Matriz de confusión

	Pred. Positivo	Pred. Negativo
Real +	TP Verdadero Positivo	FN Falso Negativo
Real -	FP Falso Positivo	TN Verdadero Negativo

FN = activo descartado → coste alto
FP = inactivo sintetizado → coste medio
En drug discovery: minimizar FN
→ aumentar recall / bajar umbral

AUC-ROC

Área bajo curva TPR vs FPR a todos los umbrales.

0.5 = azar · >0.8 = bueno
>0.9 = muy bueno

F1 Score

$F1 = 2 \cdot TP / (2 \cdot TP + FP + FN)$
Media armónica precisión/recall.

1.0 = perfecto · 0.0 = inútil
Sensible al umbral

MCC

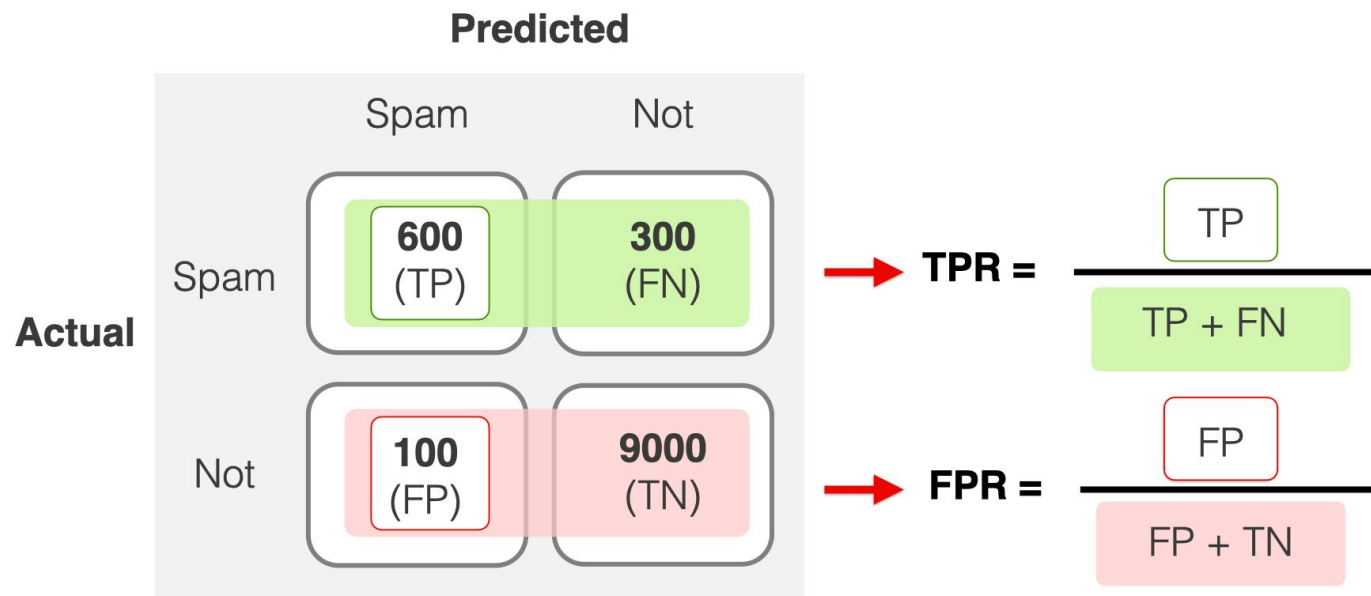
$MCC = (TP \cdot TN - FP \cdot FN) / \sqrt{((TP+FP)(TP+FN)(TN+FP)(TN+FN))}$

1=perfecto · 0=azar · -1=inverso
Más robusto al desbalance

Precision / Recall

Precisión = $TP / (TP + FP)$
Recall (sensibilidad) = $TP / (TP + FN)$

Curva PR mejor que ROC cuando hay mucho desbalance



Baseline y Regresión Logística · El punto de partida

Siempre empieza con el modelo más simple. Si no lo superas, hay un problema con los datos o el pipeline — no con el modelo.

Baseline (DummyClassifier)

Estrategia 'most_frequent': siempre predice la clase mayoritaria (inactivo).

¿Para qué sirve?

Define el piso mínimo de rendimiento.

Si tu modelo sofisticado no supera al baseline, algo está mal.

Resultado típico en QSAR:

Accuracy = 75–85% (clases desbalanceadas)

AUC-ROC = 0.50 (azar puro)

F1 = 0.00 (nunca predice activo)

Regresión Logística

Modelo lineal que estima la probabilidad de que una molécula sea activa:

$$P(\text{activo}) = \sigma(w \cdot X + b)$$

donde σ es la función sigmoide

Cada descriptor tiene un coeficiente w .

$w > 0 \rightarrow$ el descriptor favorece actividad

$w < 0 \rightarrow$ el descriptor la perjudica

Ventajas:

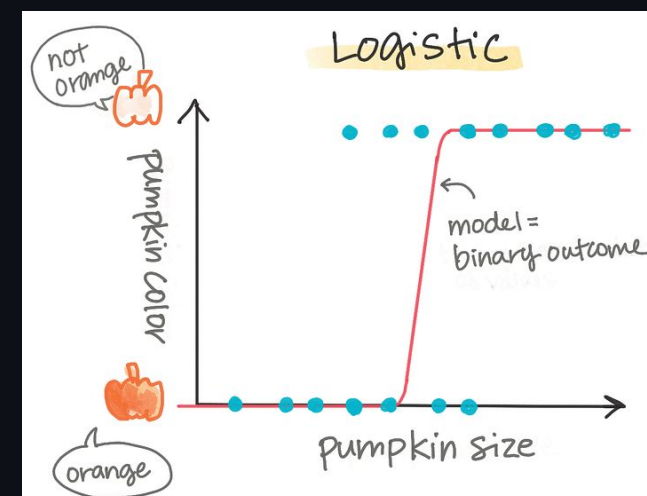
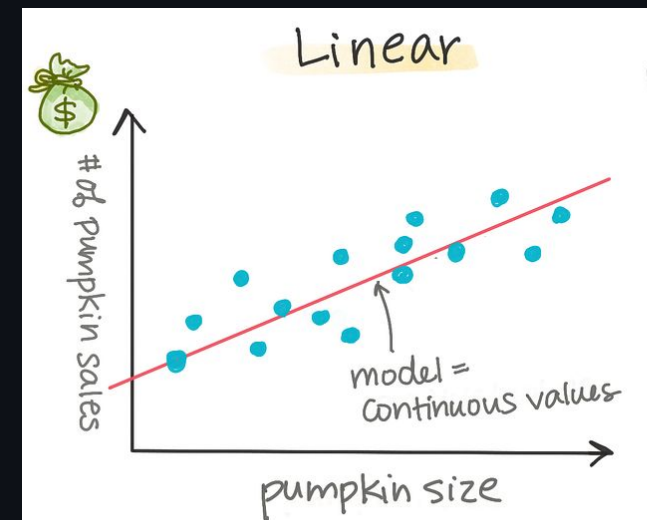
Rápido · Interpretable · Baseline sólido

Los coeficientes son proxy de importancia

Limitación:

Solo captura relaciones lineales entre descriptores y actividad.

		
14	NO	YES
15	YES	YES
16	YES	YES
17	NO	YES
18	NO	YES
19	NO	YES
20	YES	YES
21	YES	YES
22	YES	YES
23	YES	YES
24	YES	YES
25	YES	YES
26	NO	YES
27	YES	YES



Random Forest · El caballo de batalla del QSAR

Ensemble de árboles de decisión entrenados en subconjuntos aleatorios de datos y features. Robusto, interpretable y consistentemente bueno en QSAR.

¿Cómo funciona?

1. Crear N árboles de decisión
($N = n_{\text{estimators}}$, típico 100–500)

2. Cada árbol se entrena con una submuestra aleatoria de los datos (bootstrap sampling)

3. En cada nodo se evalúan solo $\sqrt{n_{\text{features}}}$ descriptores aleatorios — no todos

4. Para predecir: cada árbol vota.
La clase mayoritaria gana.
La probabilidad = fracción de votos.

Importancia de features:

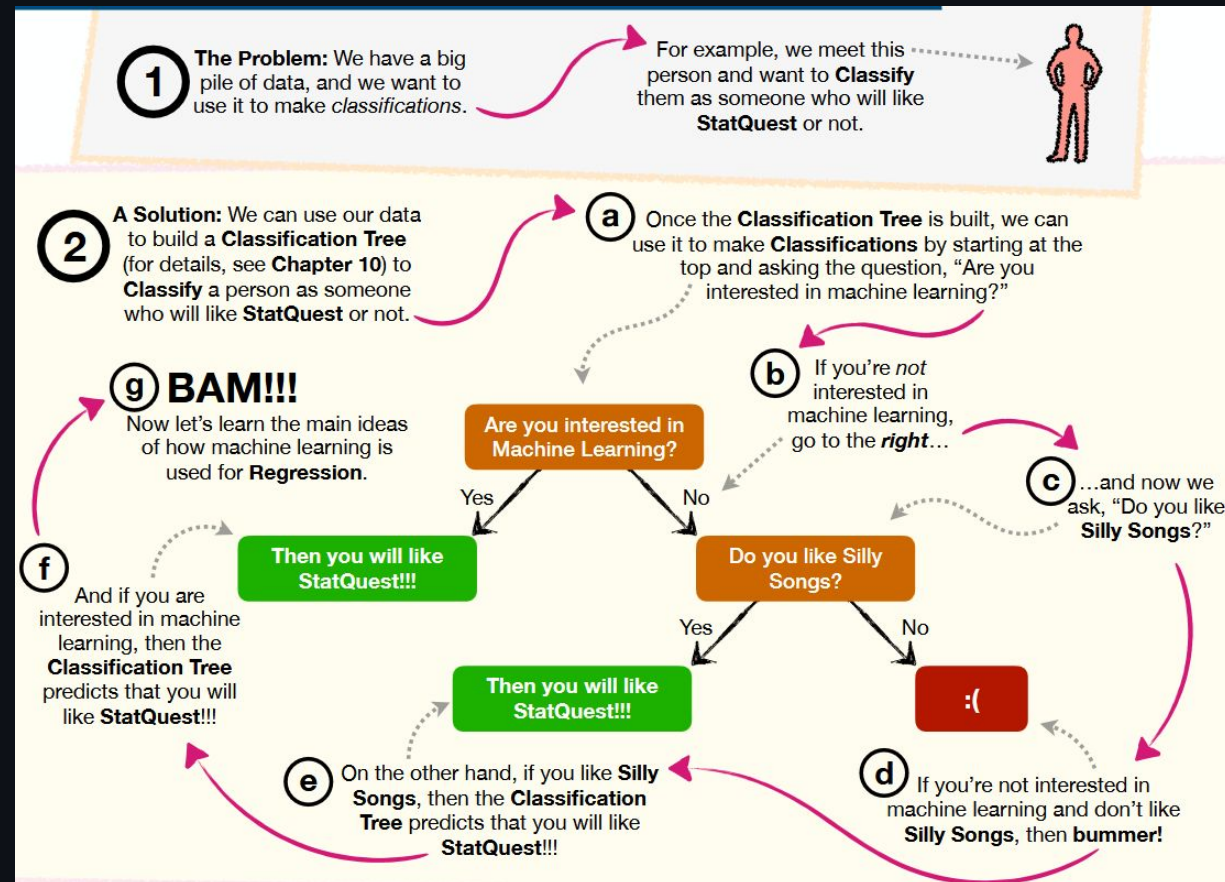
Cuánto mejora la pureza de cada nodo cuando se usa ese descriptor.
→ Feature importance directa

✓ Ventajas

No requiere escalado de features
Maneja features redundantes
Robusto al sobreajuste
Feature importance integrada
Muy bueno con fingerprints binarios

⚠ Limitaciones

Lento con muchos árboles
Puede memorizar si $n_{\text{estimators}}$ bajo
Extrapolación deficiente (no predice fuera del rango del training set)



SVM y XGBoost · Modelos alternativos en QSAR

SVM — Support Vector Machine

Encuentra el hiperplano que maximiza el margen entre las dos clases.

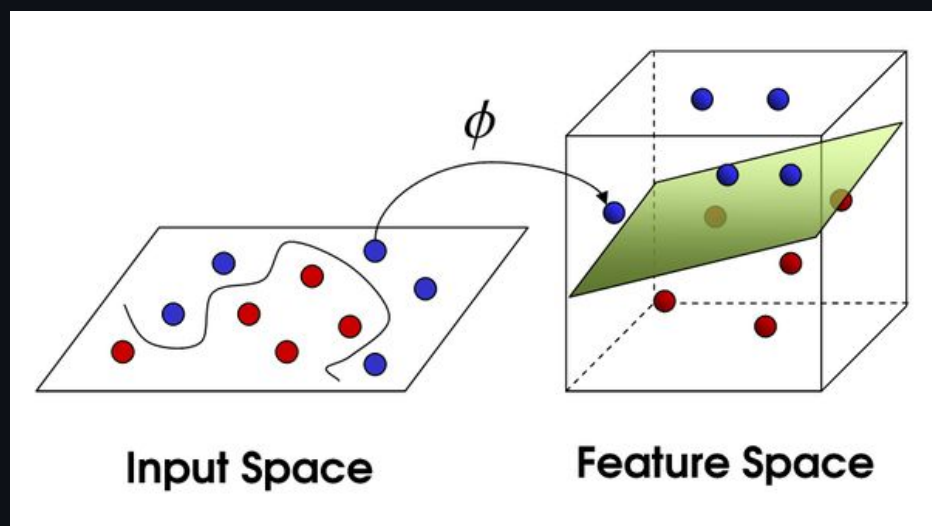
Con kernel RBF mapea los datos a un espacio de mayor dimensión donde son linealmente separables.

Hiperparámetros clave:

C → regularización (alto C = menos margen, más ajuste a los datos)

γ → radio de influencia de cada punto (alto = sobreajuste)

Históricamente el estándar en QSAR (2005–2015). Hoy superado por RF y XGBoost en la mayoría de benchmarks, pero aún válido para datasets pequeños (< 1000 moléculas).



XGBoost — Gradient Boosting

Construye árboles secuencialmente. Cada árbol corrige los errores del anterior.

Diferencia clave con Random Forest: RF: árboles independientes en paralelo (bagging — reduce varianza)

XGBoost: árboles secuenciales (boosting — reduce sesgo y varianza)

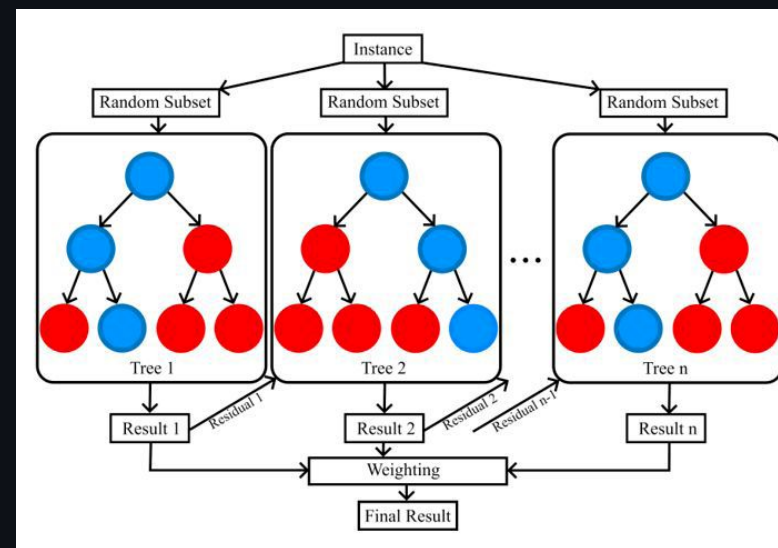
Hiperparámetros clave:

$n_estimators$ → número de árboles

$learning_rate$ → paso de aprendizaje

max_depth → profundidad de cada árbol

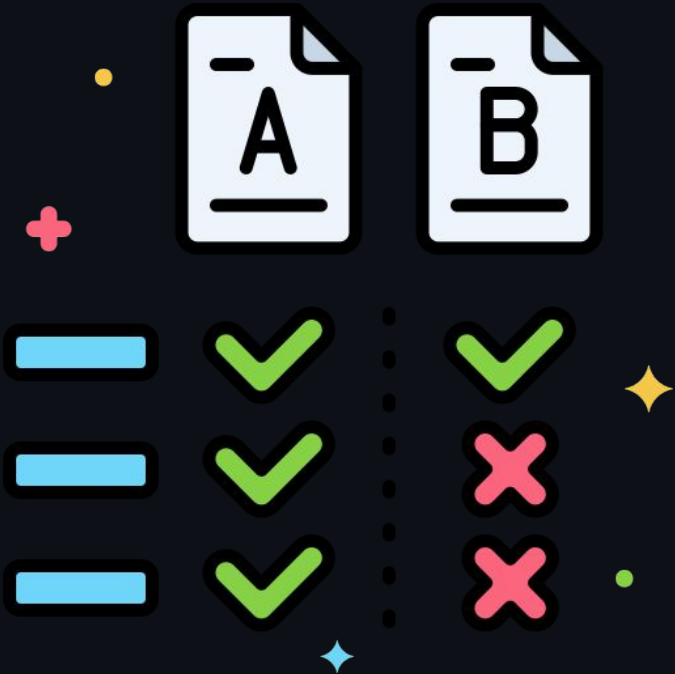
$subsample$ → fracción de datos por árbol



Comparación de Modelos · ¿Cuál elegir en QSAR?

No existe un modelo universalmente mejor. La elección depende del tamaño del dataset, la necesidad de interpretabilidad y el tiempo disponible.

Modelo	Escala	Interpretable	Sin escalado	AUC típico	Cuándo usarlo
Baseline	cualquier	✗	✓	0.50	Control negativo — siempre primero
Reg. Logística	≥100	✓	✗	0.65–0.78	Primer modelo real · coeficientes
SVM (RBF)	≤10k	✗	✗	0.72–0.82	Datasets pequeños · benchmark clásico
Random Forest	≥500	⚠	✓	0.75–0.87	Estándar actual · fingerprints · FP
XGBoost	≥1k	⚠	✓	0.78–0.90	Máximo rendimiento · más hiperparámetros



Con fingerprints Morgan (binarios): RF y XGBoost son superiores porque no asumen linealidad y manejan bien la alta dimensión dispersa (2048 bits).

Con descriptores Mordred (continuos): SVM y Regresión Logística son más competitivos.

Y-Scrambling · Validación del poder predictivo real

Si el modelo aprende patrones reales, su rendimiento debe caer drásticamente cuando las etiquetas se mezclan al azar. Si no cae, hay un problema.

¿Qué es y por qué importa?

El Y-scrambling (o Y-randomization) consiste en entrenar el modelo múltiples veces con las etiquetas (y) mezcladas aleatoriamente.

Resultado esperado:

AUC real $\sim 0.80 \leftarrow$ modelo aprende

AUC con y random $\sim 0.50 \leftarrow$ solo azar

Si AUC con y random > 0.65 :

- ⚠ El modelo está memorizando el dataset
- ⚠ Hay data leakage (filtración de info del test set al training set)
- ⚠ Los features contienen info de y de forma implícita (ej: calcular features sobre el dataset completo antes del split)

Estándar regulatorio en QSAR:

OECD Principle 4 lo exige explícitamente para modelos enviados a agencias.

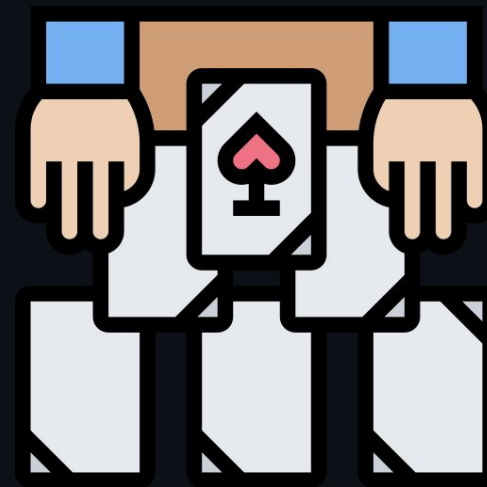
¿Cómo interpretar el resultado?

Visualizar como histograma: Distribución de AUC con y random (rojo) + línea vertical del AUC real (verde)

El modelo es válido si:

AUC real $\gg$ media de scrambles

Y la línea verde está fuera de la distribución roja ($p < 0.05$)



Dominio de Aplicabilidad · ¿Cuándo confiar en la predicción?

Un modelo QSAR es confiable solo dentro del espacio químico que cubrió durante el entrenamiento. Fuera de ese espacio, la predicción no tiene respaldo estadístico.

¿Qué es el dominio de aplicabilidad?

El subespacio del espacio químico donde el modelo tiene evidencia suficiente para hacer predicciones confiables.

Analogía simple:

Un modelo entrenado solo con inhibidores de quinasas no debería predecir la actividad de antibióticos — aunque técnicamente puede dar un número.

Tres formas de medirlo:

1. Distancia al centroide del training set Si la molécula nueva está muy lejos del centro del espacio de entrenamiento, la predicción es poco confiable.
2. Leverage (hat matrix): Estadístico clásico en QSAR regulatorio. Umbral: $h > 3k/n$ (k =features, n =muestras)
3. Nearest neighbor distance: Si la molécula más similar en training tiene Tanimoto < 0.4 , fuera del dominio.



Para el proyecto final

Cuando predigan sobre moléculas nuevas de ChEMBL o del docking, siempre reportar:

1. La predicción (activo/inactivo, pIC50)
2. Si la molécula está dentro del dominio
3. La distancia al centroide del training

Una predicción fuera del dominio no es necesariamente incorrecta, pero debe marcarse como 'extrapolación'.

